TECHNICAL MANUSCRIPT

A New Prediction Algorithm Based on the Takagi–Sugeno Fuzzy Model and an Improved Shark Smell Optimization Technique and Its Application

Revised, reconstructed, and equation-enabled edition

— August 2026 —

Enhanced figures · native Word equations · repaired reference-code appendices

Abstract

This paper proposes T-SFSSO (Takagi–Sugeno fuzzy shark smell optimization), a new time-series prediction algorithm that couples dynamic Takagi–Sugeno (T-S) fuzzy modelling with an improved shark smell optimization algorithm, referred to as fuzzy shark smell optimization (FSSO). The conventional shark smell optimization (SSO) algorithm converges slowly and is prone to premature convergence at local optima, which limits its usefulness for parameter identification of fuzzy prediction models. FSSO removes these weaknesses through four coordinated modifications. First, the time interval Δt_m that governs the scope of the m-th forward movement — fixed to 1 in previous studies — is adapted online by a probabilistic rule driven by the magnitude of the gradient, which markedly accelerates convergence. Second, a gradient-triggered jump mechanism relocates sharks that stagnate in the basin of a local optimum, strengthening the global search. Third, the upper bound of the velocity-limit constant is corrected: it is drawn from the interval (0, 2) instead of (0, 1), which removes the systematic speed decay that froze earlier SSO variants. Fourth, the rotational-movement coefficient is generated on (−1.3, 1.3) rather than (0, 1), so the local search can reverse sign and reach regions that were previously inaccessible. The convergence, stability, and computational complexity of the resulting algorithm are analyzed formally. In benchmark experiments, FSSO outperforms the conventional SSO, its chaotic and normal-distribution variants, and PSO, GA, DE, FACS, and ISOA; in particular, FSSO discovered the negative-valued region of a weighted-sum benchmark function whose minimum had previously been reported as zero, and this discovery is corroborated here by a closed-form proof that the true minimum equals $-w/(4(w+1)) \approx -0.2499$. Finally, a dynamic T-S fuzzy model whose parameters are initialized by recursive least squares and refined by FSSO is applied to the prediction of daily confirmed COVID-19 cases in seven countries. The proposed T-SFSSO attains a training MAPE of 2.74% against 4.68% for the recent FTSOAX model, a test MAPE of 4.93% against 7.82%, and reduces RMSE by roughly one half, achieving the best RMSE and SMAPE among all compared models in every country examined. Python and MATLAB reference implementations of all algorithms are provided in the appendices.

Keywords — Takagi–Sugeno fuzzy model; shark smell optimization; swarm intelligence; parameter identification; time-series prediction; epidemic forecasting; convergence analysis

Upgrade applied

All 43 display equations are rebuilt as editable Word mathematics. Source figures 1–8 are retained at their strongest available resolution; blurred figures 9–14 are replaced with clean reconstructions. The incomplete code appendices in the source export are repaired with complete reference implementations.

1. Introduction

Accurate prediction of the number of newly infected persons is a prerequisite for timely public-health decision making, hospital-capacity planning, and the assessment of intervention policies. A large body of work has therefore addressed epidemic time-series forecasting from complementary directions. Comprehensive treatments of predictive decision support during the COVID-19 crisis are given in [1]. Compartmental approaches remain a natural starting point: SEIR-type models were used for trend prediction in [2], a fractional SEIR model with data-driven prediction of Omicron-variant dynamics was developed in [3], and a combined SEIR–ARIMA scheme was applied to the epidemic in Kenya in [4]. Purely statistical and hybrid statistical–neural methods have also proven effective: a hybrid ARIMA–neural-network model for worldwide daily case counts was proposed in [5], Bayesian structural time-series models combined with ARIMA were used to quantify the causal impact of vaccination in [6], and ARIMA/SARIMA models were systematically fitted to the cumulative-case

dynamics of sixteen countries in [7]. On the recurrent-network side, an LSTM-based framework (r-LSTM) for confirmed-case forecasting was presented in [8], building on the long short-term memory architecture of [9].

More recent studies, published between 2022 and 2026, sharpen the picture of what data-driven epidemic forecasting can and cannot achieve. Comparative analyses of GRU, LSTM, ARIMA, and SARIMA cells in [10] and transfer-learning LSTM models in [11] show that recurrent architectures generalize well across countries but remain sensitive to distribution shift; incorporating variant-frequency covariates into deep forecasters measurably improved short-term accuracy in the USA [12]. At the systems level, the large-scale evaluation of individual and ensemble probabilistic forecasts in [13] and the retrospective assessment of prospective modelling studies in [14] both emphasize that calibrated, frequently re-identified models outperform static ones — an observation that directly motivates the fast re-identification capability pursued in this paper. Surveys of COVID-19 forecasting models [15] and of machine-learning and deep-learning diagnosis models [16] summarize the state of the art and consistently list interpretability and training cost among the open problems of purely deep approaches. Fuzzy time-series (FTS) methods form a third line of research that combines transparency with strong nonlinear modelling capability, and intelligent optimization algorithms have been the main engine of their recent progress. Particle swarm optimization (PSO) was used to tune partitions and fuzzy order for COVID-19 forecasting in [17], and a grey rolling model optimized by PSO was applied to short-term epidemic prediction in [18]. An improved seagull optimization algorithm (ISOA) coupled with XGBoost — the FTSOAX model — advanced FTS forecasting of daily confirmed cases in [19]. Earlier weighted FTS models for exchange-rate forecasting [20], hybrid FTS systems with multi-objective optimization for wind speed [21], FTS models with GA-based fuzzy c-means and multifactor BPNN for stock forecasting [22], refined exponentially weighted FTS with improved harmony search for load forecasting [23], and hybrid FTS models for Indian COVID-19 case and death prediction [24] all follow the same recipe: a fuzzy forecasting skeleton whose free parameters are identified by a metaheuristic. The trend continues in the most recent literature — a mixed-order intuitionistic FTS method trained by a new artificial neural network and grey wolf optimization appeared in 2026 [25], and boosted ANFIS models tuned by an augmented marine predator algorithm were proposed for renewable-energy forecasting in [26]. These works demonstrate that the accuracy of a fuzzy predictor is decided largely by the quality of the optimizer that identifies it.

Fuzzy inference models themselves divide into the Mamdani type [27] and the Takagi–Sugeno (T-S) type [28]. The Mamdani model represents fuzzy rules constructed from expert experience and composes a linguistic consequent through fuzzy inference; consequently the number of rules can grow exponentially during modelling, the computational burden of inference is high, and the linguistic consequent complicates system design. In contrast, because the consequent part of a T-S model is a linear function, the T-S model can be treated with standard linear-systems theory, its stability can be certified, and complex nonlinear systems can be modelled accurately with far fewer rules [29, 30]. For these reasons the T-S model is preferred in practice, and the key to its performance is system identification and parameter learning. Gradient-descent and least-squares-estimation methods were employed in [31, 32], while evolutionary and swarm methods — chaotic PSO [34], genetic algorithms [35] — brought global-search capability to the identification problem; ANFIS-style neuro-fuzzy learning [31] and modern TSK learning algorithms with non-separable rules [36] continue this development, and neuro-fuzzy inference systems have been applied directly to COVID-19 diagnosis [37]. Evolving parameter-optimization schemes for online prediction of time series with changing dynamics [33] confirm that fast, repeatable identification is the bottleneck. The optimizer landscape available for such identification has expanded enormously. Beyond the classical particle swarm optimization [38], genetic algorithm [39], differential evolution [40], and fuzzy ant colony system [41], the years 2022–2024 alone produced the reptile search algorithm [42], dwarf mongoose optimization [43], prairie dog optimization [44], snake optimizer [45], dandelion optimizer [46], artificial hummingbird algorithm [47], artificial rabbits optimization [48], golden jackal optimization [49], pelican optimization [50], starling murmuration optimizer [51], Ebola

optimization search [52], the INFO weighted-mean-of-vectors method [53], coati optimization [54], the Kepler [55] and nutcracker [56] optimizers, sand cat swarm optimization [57], the physics-based RIME method [58], gazelle optimization [59], the hippopotamus optimization algorithm [60], and the parrot optimizer [61]. This proliferation has been accompanied by calls for greater rigor in benchmarking [62] and by renewed theoretical interest in the convergence guarantees of swarm methods [63]. Within this landscape, the shark smell optimization (SSO) algorithm [64] occupies a distinctive position: unlike most swarm methods, it exploits gradient information of the objective function — imitating a shark that follows an intensifying smell of blood — in addition to stochastic rotational (local) search. Extensions include the chaotic variant CSSO used for short-term wind-power prediction [65] and a multi-objective SSO with normally distributed coefficients (NSSO) for automatic train operation [66]; SSO has recently been improved and applied to human action recognition [67], while shark-inspired designs such as the white shark optimizer [68] and the Genghis Khan shark optimizer [69] confirm the continued interest in this family.

Despite these merits, the conventional SSO suffers from four structural weaknesses that this paper identifies and repairs. (i) The time interval Δt_m that scales each forward movement is fixed to 1, so the step length ignores how steep the objective currently is; convergence is unnecessarily slow whenever the colony is far from the optimum. (ii) Once a shark enters the basin of a local optimum, only the rotational movement remains as an escape mechanism, and with a rotation coefficient confined to (0, 1) the rotation cannot change the sign of any coordinate — the shark is effectively trapped. (iii) The speed-limit constant γ_m is restricted to (0, 1), which makes the attainable speed non-increasing across stages, so the colony can freeze far from the optimum (a defect verified experimentally in Section 5). (iv) The random coefficients used for the rotational movement were generated on intervals whose bounds were never justified, and the published choice prevents the discovery of solutions with negative coordinates — which, as Section 5 proves, caused previous studies to misreport the minimum of a standard benchmark as zero. Motivated by these observations, this paper makes the following contributions: 1. An improved SSO algorithm (FSSO). Four coordinated modifications — a probabilistic, gradient-driven adaptive time interval; a gradient-triggered jump for escaping local basins; a stochastic speed-limit constant $\gamma_m \sim U(0, 2)$; and an extended rotational coefficient $R_3 \sim U(-1.3, 1.3)$ — are formalized in Eqs. (26)–(32) and assembled into Algorithm 3. 2. Theory. Convergence and stability of FSSO are analyzed as dedicated subsections (Sections 3.4 and 3.5), and the computational complexity of every algorithm in the paper is derived (Section 3.6). 3. A corrected benchmark analysis. A closed-form proposition (Section 5.1) proves that the weighted-sum benchmark f_1 attains the negative minimum $-w/(4(w+1)) \approx -0.2499$, corroborating the experimental discovery made by FSSO and correcting the value 0 reported in earlier studies. 4. T-SFSSO. A complete prediction pipeline (Section 4, Algorithm 4) in which a dynamic T-S fuzzy model is structure-identified, initialized by recursive least squares, and refined by FSSO; applied to daily confirmed COVID-19 cases of seven countries it halves the RMSE of the recent FTSOAX model [19] and attains the best RMSE and SMAPE in all seven countries. 5. Reproducibility. Complete Python and MATLAB reference implementations of all algorithms, together with regenerated convergence figures and three-dimensional visualizations of every benchmark function, are provided (Appendices A and B). The remainder of the paper is organized as follows. Section 2 reviews the dynamic T-S fuzzy model and the recursive-least-squares identification that serves as the baseline. Section 3 reviews the conventional SSO, develops the four improvements, states the FSSO algorithm, and analyzes its convergence, stability, and complexity. Section 4 presents the T-SFSSO prediction pipeline. Section 5 reports benchmark experiments, including the closed-form analysis of f_1. Section 6 applies T-SFSSO to epidemic data from seven countries. Section 7 discusses strengths, limitations, and future research directions, and Section 8 concludes.

2. Preliminaries

2.1 The dynamic T–S fuzzy model

The dynamic T–S fuzzy model with one output and p inputs is described by the rule base [28, 33].

$$R^i: IF\ y(k-1)\ is\ A_1^i, \ldots, u_p(k-t_{dp}-n_p)\ is\ A_m^i, THEN\ y^i(k)=a_0^i+a_1^i y(k-1)+\cdots+a_m^i u_p(k-t_{dp}-n_p) \quad (1)$$

Here R^i is the i-th fuzzy rule, A_j^i is its j-th fuzzy subset, $y^i(k)$ is the consequent value, and a_j^i is a consequent parameter. The variables $u_1(\cdot)$, ..., u_p(·) are inputs, y(·) is the output, and k is the discrete time index. The lag orders are n_y and n_j, and t_dj denotes transport delay.

Defining the regression (state) variables

$$x_1(k)=y(k-1),\ldots,x_{n_y}(k)=y(k-n_y),x_{n_y+1}(k)=u_1(k-t_{d1}),\ldots,x_m(k)=u_p(k-t_{dp}-n_p) \quad (2)$$

collects all lagged outputs and inputs into a regression vector of dimension

$$m=n_y+\sum_{i=1}^{p}(n_i+1) \quad (3)$$

so that the rule base can be written compactly as

$$R^i: IF\ x_1\ is\ A_1^i, \ldots, x_m\ is\ A_m^i, THEN\ y^i=a_0^i+a_1^i x_1+\cdots+a_m^i x_m \quad (4)$$

In the dynamic model, data at time k are affected by preceding values through the lagged regressors. Parameter learning is therefore central. The modelling error is measured by root-mean-square error

$$E=\sqrt{\frac{1}{T}\sum_{k=1}^{T}(y(k)-\hat{y}(k))^2} \quad (5)$$

where T is the number of samples, ŷ(k) is the model output, and y(k) is the measured output. For input vector (x_{1t}, ..., x_m_t), the prediction is the weighted average of rule consequents

$$\hat{y}_t=\frac{\sum_{i=1}^{n}G_t^i y_t^i}{\sum_{i=1}^{n}G_t^i}=\frac{\sum_{i=1}^{n}G_t^i(a_0^i+a_1^i x_{1t}+\cdots+a_m^i x_{mt})}{\sum_{i=1}^{n}G_t^i} \quad (6)$$

with firing strength

$$G_t^i=\prod_{k=1}^{m}A_k^i(x_{kt}) \quad (7)$$

Triangular membership functions are used throughout. The regressor and parameter vectors are

$$H_t=(G_t^1,G_t^1 x_{1t},\ldots,G_t^1 x_{mt},\ldots,G_t^n,G_t^n x_{1t},\ldots,G_t^n x_{mt}) \quad (8)$$

$$\theta=(a_0^1,a_1^1,\ldots,a_m^1,a_0^2,\ldots,a_m^n) \quad (9)$$

Stacking the samples yields the linear regression

$$Y=\hat{Y}+E_r=H\theta^T+E_r \quad (10)$$

where E_r is the residual vector. Because the model is linear in θ, least squares applies; the recursive form avoids recomputing a batch solution when samples arrive sequentially.

2.2 Baseline identification by recursive least squares

The baseline identifies consequent parameters by recursive least squares (RLS). The normalized firing strengths define

$$H_k=\left(W_k^1,W_k^1x_{1k},\ldots,W_k^nx_{mk}\right),W_k^j=\frac{G_k^j}{\sum_{i=1}^{n}G_k^i} \quad (11)$$

Initialize $\theta_0 = 0$ and $S_0 = \alpha I$ with $\alpha \approx 10^5$. For each new sample, update the gain, covariance, and parameter estimate:

$$F_k=\frac{S_{k-1}H_k^T}{1+H_kS_{k-1}H_k^T} \quad (12)$$

$$S_k=S_{k-1}-F_kH_kS_{k-1} \quad (13)$$

$$\theta_k=\theta_{k-1}+F_k\left(y_k-H_k\theta_{k-1}^T\right) \quad (14)$$

One full pass costs $O(L\cdot d^2)$ for $d = n(m+1)$ parameters. RLS is fast but local; it supplies the initial estimate that FSSO refines globally.

Algorithm 1. RLS identification of the dynamic T–S consequent parameters

Input: data {(x₁k,...,xmk,yk)}, k=1...L; rules n; α≈10⁵
$\theta_0 \leftarrow 0$; $S_0 \leftarrow \alpha I$
for k = 1...L:
compute G_k^i and normalized weights W_k^i; assemble H_k
$F_k \leftarrow S_{k-1}H_k^T / (1 + H_kS_{k-1}H_k^T)$
$S_k \leftarrow S_{k-1} - F_kH_kS_{k-1}$
$\theta_k \leftarrow \theta_{k-1} + F_k(y_k - H_k\theta_{k-1}^T)$
return θL

3. An Improved Shark Smell Optimization Algorithm (FSSO)

SSO exploits objective-function gradients together with stochastic rotational search. This section reviews the conventional algorithm, develops four coordinated improvements, and analyzes convergence, stability, and complexity.

3.1 The conventional shark smell optimization algorithm

3.1.1 Initialization

An initial colony of NP sharks represents candidate solutions in an ND-dimensional domain.

$$\left[X_1^1,X_2^1,\ldots,X_{NP}^1\right),NP=populationsize \quad (15)$$

$$X_i^1=\left[x_{i,1}^1,x_{i,2}^1,\ldots,x_{i,ND}^1\right),i=1,\ldots,NP \quad (16)$$

$$\left[V_1^1,V_2^1,\ldots,V_{NP}^1\right) \quad (17)$$

$$V_i^1=\left[v_{i,1}^1,v_{i,2}^1,\ldots,v_{i,ND}^1\right),i=1,\ldots,NP \quad (18)$$

3.1.2 Search step

At stage m, each shark follows the objective gradient; inertia incorporates the previous velocity; a speed cap bounds motion; forward movement is followed by local rotation and greedy selection.

$$V_i^m = \mu_m R_1 \nabla(OF)|_{X_i^m}, i = 1, \ldots, NP, m = 1, \ldots, M \quad (19)$$

$$v_{i,j}^m = \mu_m R_1 \frac{\partial(OF)}{\partial x_j}|_{x_{i,j}^m} \quad (20)$$

$$\tilde{v}_{i,j}^m = \mu_m R_1 \frac{\partial(OF)}{\partial x_j}|_{x_{i,j}^m} + \alpha_m R_2 v_{i,j}^{m-1} \quad (21)$$

$$v_{i,j}^m = sgn(\tilde{v}_{i,j}^m) min(|\tilde{v}_{i,j}^m|, \gamma_m |v_{i,j}^{m-1}|) \quad (22)$$

$$Z_i^{m+1} = X_i^m + V_i^m \Delta t_m \quad (23)$$

$$\Gamma_i^{m+1,l} = Z_i^{m+1} + R_3 \circ Z_i^{m+1} \quad (24)$$

$$X_i^{m+1} = arg\,max\{OF(Z_i^{m+1}), OF(\Gamma_i^{m+1,1}), \ldots, OF(\Gamma_i^{m+1,L})\} \quad (25)$$

Algorithm 2. Conventional shark smell optimization (SSO)

Input: OF, domain D, NP, M, L, {μm}, {αm}, {γm}
initialize X_i^1 and V_i^1
for m = 1…M, for i = 1…NP:
compute $\nabla OF(X_i^m)$; update and cap V_i^m
$Z_i^{m+1} \leftarrow X_i^m + V_i^m$ (Δt=1)
generate L rotations with $R_3 \sim U(0,1)$
greedily retain the best candidate
return best solution

3.2 The four proposed improvements

3.2.1 Adaptive timing of the forward movement

A large gradient signals a steep region and justifies a longer forward interval. The deterministic and probabilistic rules are

$$\Delta t_m = \begin{cases} 1, & G < C_g \\ (G/C_g) C_t R_4, & G \geq C_g \end{cases} \quad (26)$$

$$\Delta t_m = \begin{cases} 1, & G < C_g \\ FM, & G \geq C_g \end{cases} \quad (27)$$

$$FM = \begin{cases} 1, & q < Q \\ (G/C_g) C_t R_4, & q \geq Q \end{cases} \quad (28)$$

With probability Q, the unit step is retained; otherwise, the interval grows proportionally to G/C_g. This preserves short steps near stationary points and accelerates high-gradient traverses.

3.2.2 Gradient-triggered jump

A shark that repeatedly sees a near-zero gradient can be trapped in a local basin. At periodic stages, the indicator and relocation are

$$J_i^m = \begin{cases} 1, & \| \nabla OF(X_i^m) \| < \varepsilon_J \; and \; m \equiv 0 (mod \, T_J) \\ 0, & otherwise \end{cases} \quad (29)$$

$$Z_i^{m+1} = X_{lo} + (X_{hi} - X_{lo}) \circ U(0,1)^{ND} \; whenever \; J_i^m = 1 \quad (30)$$

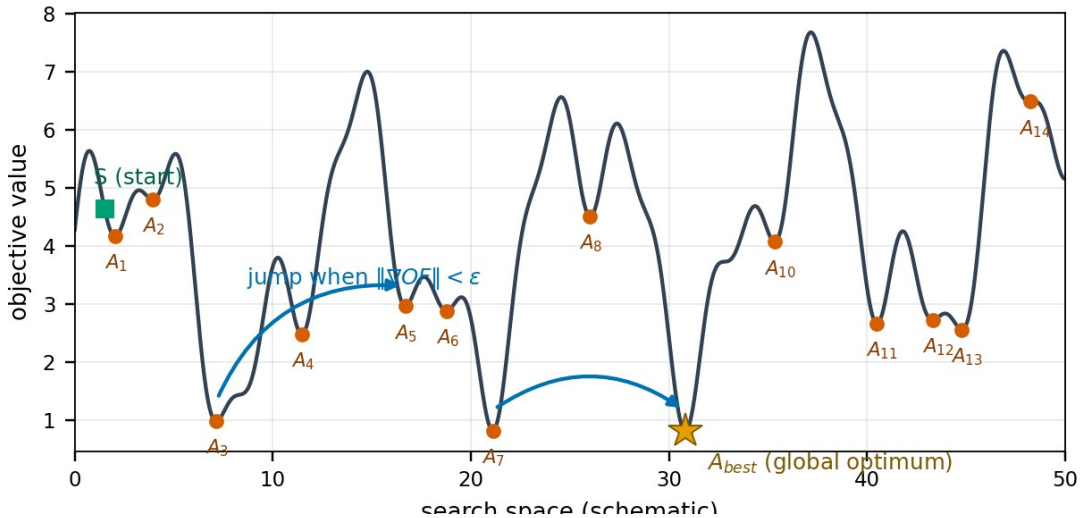


Fig. 1. Optimization landscape with local optima and the gradient-triggered jump mechanism (publication-quality source figure).

3.2.3 The shark's speed limitation

When γ_m is restricted to (0,1), an active cap forces geometric speed decay and can freeze the colony away from the optimum. FSSO uses a neutral stochastic cap with equal probability of acceleration and deceleration:

$$\gamma_m \sim U(0,2) \; independently \; redrawn \; at \; every \; stage \; m \quad (31)$$

3.2.4 Sign-reversing rotational search

The conventional $R_3 \in (0,1)$ preserves coordinate signs. The extended interval makes sign reversal possible while keeping the search local:

$$\Gamma_i^{m+1,l} = Z_i^{m+1} + R_3 \circ Z_i^{m+1}, R_3 \sim U(-1.3, 1.3)^{ND} \quad (32)$$

3.3 The proposed FSSO algorithm

Algorithm 3. Proposed fuzzy shark smell optimization

Input: OF, D, NP, M, L, μ, α, Cg, Ct, Q, εJ, Tjump
initialize positions and small random velocities
for each stage m: draw γm ~ U(0,2)
compute g = ∇OF and G = ‖g‖; update and hard-cap velocity
choose Δtm from the probabilistic adaptive rule
update forward candidate; apply periodic jump when triggered
create L rotations with R_3 ~ U(−1.3,1.3)ND
greedily retain the best candidate
return the best solution

For minimization, set OF = −f so that maximizing OF is equivalent to minimizing f.

3.4 Convergence analysis

Expected velocity is bounded by the contraction

$$E \parallel V^m \parallel \leq \frac{\acute{\mu}\acute{G}}{2} + \frac{\acute{\alpha}}{2} E \parallel V^{m-1} \parallel \quad (33)$$

$$\rho := \acute{\alpha}/2 < 1 \Rightarrow \limsup_{m \to \infty} E \parallel V^m \parallel \leq \frac{\acute{\mu}\acute{G}/2}{1 - \acute{\alpha}/2} \quad (34)$$

Greedy best-so-far retention makes the objective sequence monotone

$$f\left(X^{best,m+1}\right) \leq f\left(X^{best,m}\right), m = 1, 2, \ldots \quad (35)$$

Jumps give every positive-measure neighbourhood of a global minimizer a nonzero visitation probability. With continuity and monotonicity, this yields almost-sure convergence of the best value

$$P\left(X^{m+1} \in B \mid F_m\right) \geq \delta_B > 0 \Rightarrow P\left(\lim_{m \to \infty} f\left(X^{best,m}\right) = f^*\right) = 1 \quad (36)$$

Thus FSSO combines geometric expected-velocity contraction with irreducible global exploration. Conventional SSO lacks the jump and sign-reversing rotation needed for the latter.

3.5 Stability analysis

Hard velocity clipping and projection to compact D keep every search state bounded:

$$\parallel X_i^{m+1} - X_i^m \parallel_\infty \leq v_{max} \Delta t_{max} < \infty, X_i^m \in D \; \forall \, m \quad (37)$$

Normalized nonnegative firing strengths make the predictor output a convex combination of rule consequents:

$$\min_{1 \leq i \leq n} y_t^i \leq \hat{y}_t \leq \max_{1 \leq i \leq n} y_t^i \quad (38)$$

For bounded regressors the one-step prediction is bounded. Rule-wise Schur stability provides standard dynamic T–S stability certification.

3.6 Computational complexity

$$C_{FSSO} = O\left(M \cdot NP \cdot \left(c_\nabla + L c_f + N D\right)\right), c_\nabla = 2 N D c_f \quad (39)$$

With numerical gradients the order is O(M·NP·ND·c_f), linear in each design dimension, and memory is O(NP·ND). The four improvements add only constant-order random draws and comparisons.

4. Realization of the Proposed T–SFSSO Prediction Method

T–SFSSO combines a dynamic T–S fuzzy model, RLS initialization, and FSSO refinement. RLS first produces a computationally cheap, well-posed estimate; FSSO then refines antecedent and consequent parameters globally.

1. Data and fuzzification. Split daily cases into 80% training and 20% test data; cover each input with triangular membership functions.
2. Structure identification. Combine the fuzzy input sets to form the initial rule base.
3. RLS initialization. Estimate consequent parameters using Algorithm 1.
4. FSSO refinement. Initialize the colony near θ_RLS (one elite at θ_RLS) and minimize training RMSE.
5. Model assembly. Combine the identified structure and optimized parameters; evaluate the final prediction by weighted-average defuzzification.

The relative absolute training error used for monitoring is

$$E_{rel}=\sum_{t=1}^{L_{tr}}\frac{\left|y_t-\hat{y}_t\right|}{y_t} \quad (40)$$

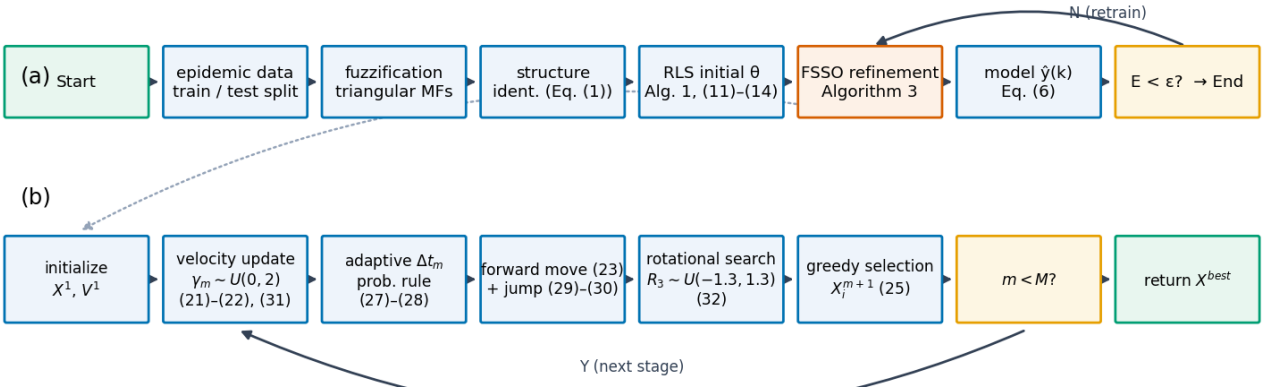


Fig. 8. Flow chart of the T–SFSSO prediction pipeline and the inner FSSO loop (publication-quality source figure).

Algorithm 4. Complete T–SFSSO prediction procedure

Input: time series; 80/20 split; n rules; lags and delays; FSSO parameters
build triangular membership functions and the dynamic T–S rule base
θRLS ← RLS identification on training data
initialize FSSO around θRLS; θ* ← FSSO minimizing training RMSE
assemble model; repeat refinement if the stopping error is not met
predict by weighted-average T–S output; evaluate RMSE, MAPE, SMAPE

5. Benchmark Experiments and Analysis

Two experiment groups compare FSSO with previous SSO variants and with established optimizers. Unless stated otherwise, NP = 10 and M = 50; reported table values come from the original campaign, while convergence figures were regenerated from reference implementations.

5.1 Comparison with previous SSO algorithms

Table 1. Standard test functions of the first experiment

Name	Function	Domain	Minimum
Function-1 (f_1)	$\Sigma x_i^2 + (\Sigma 0.5ix_i)^2 + \Sigma 0.5ix_i$	[−5,30], n=30	−0.2499 (proved)
Function-2 (Matyas)	$0.26(x^2+y^2) - 0.48xy$	[−10,10], n=2	0

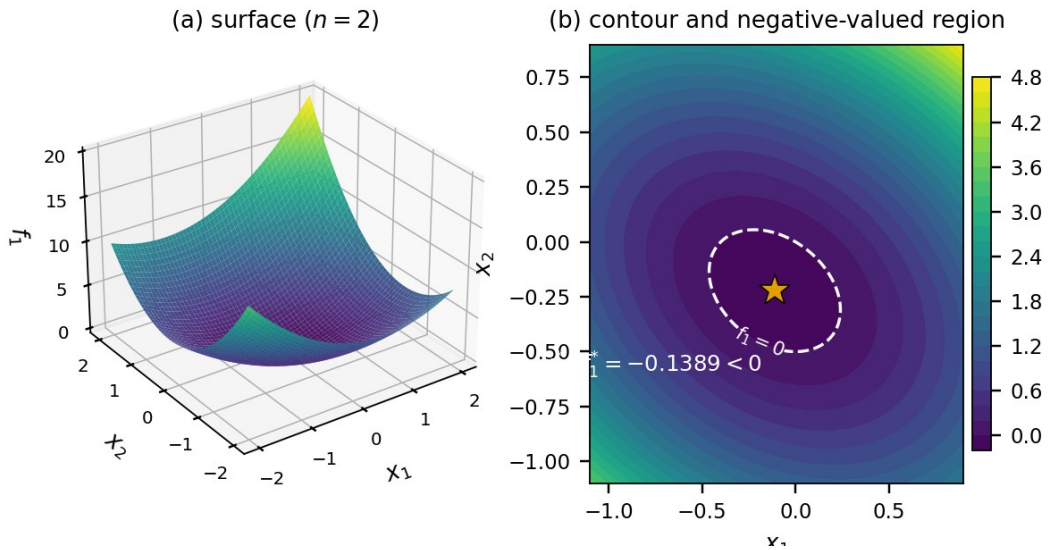


Fig. 2. Function-1 surface and contour for n = 2; dashed zero-level set and analytic minimizer.

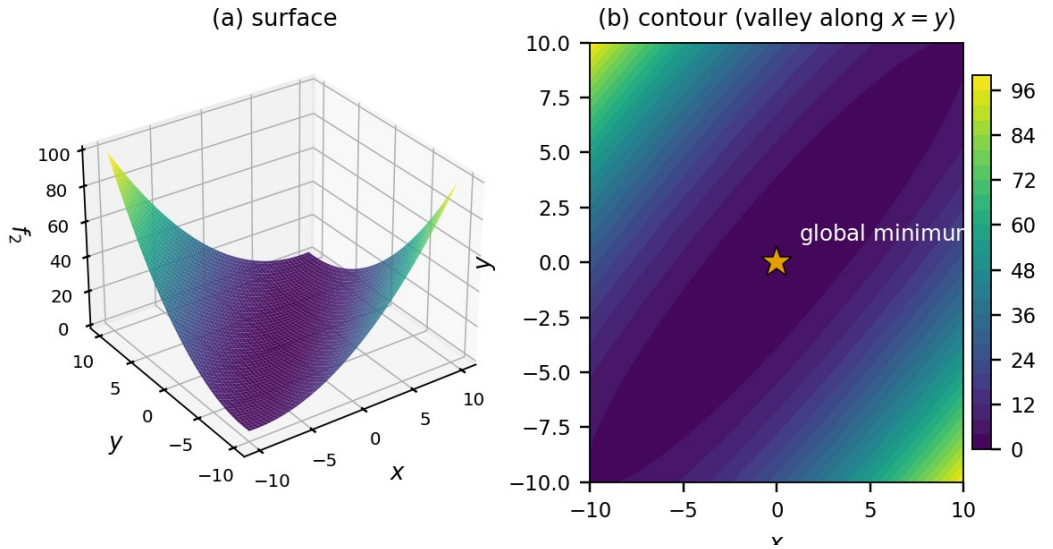


Fig. 3. Matyas-function surface and contour with global minimum at the origin.

Table 2. Comparison with previous SSO algorithms

Algorithm	Statistic	f_1	f_2
SSO	maximum	3.78×10^{-5}	0.62×10^{-6}
SSO	minimum	1.56×10^{-13}	3.1×10^{-27}
SSO	mean	5.13×10^{-6}	1.52×10^{-8}
SSO	std.	4.98×10^{-6}	1.51×10^{-7}
SSO	runtime (s)	0.636	0.478
CSSO	maximum	4.43×10^{-7}	2.16×10^{-8}
CSSO	minimum	8.99×10^{-14}	1.56×10^{-27}
CSSO	mean	3.75×10^{-9}	5.23×10^{-10}
CSSO	std.	6.27×10^{-8}	3.79×10^{-9}
CSSO	runtime (s)	0.672	0.401
NSSO	maximum	7.32×10^{-5}	0.34×10^{-6}
NSSO	minimum	1.14×10^{-14}	3.5×10^{-27}
NSSO	mean	3.68×10^{-6}	7.42×10^{-9}
NSSO	std.	7.64×10^{-5}	0.55×10^{-7}
NSSO	runtime (s)	0.601	0.483
FSSO	maximum	−0.247	0

Algorithm	Statistic	f_1	f_2
FSSO	minimum	−0.249	0
FSSO	mean	−0.2483	0
FSSO	std.	1.59×10^{-5}	0
FSSO	runtime (s)	0.593	0.423

Proposition 1 — closed-form minimum of f_1

Let $c = (0.5, 1.0, \ldots, 0.5n)$ and $w = \|c\|^2$. The strictly convex function $f_1(x) = \|x\|^2 + (c^T x)^2 + c^T x$ has the unique minimizer and minimum

$$x^* = -\frac{c}{2(w+1)}, f_1^* = -\frac{w}{4(w+1)}, w = \|c\|^2 = \sum_{i=1}^{n} (0.5\,i)^2 \quad (43)$$

For $n = 30$, $w = 2363.75$ and $f_1^* = -0.249894\ldots$. Strict convexity follows from the positive-definite Hessian $2(I + cc^T)$; solving $\nabla f_1 = 0$ yields the stated x^* and minimum.

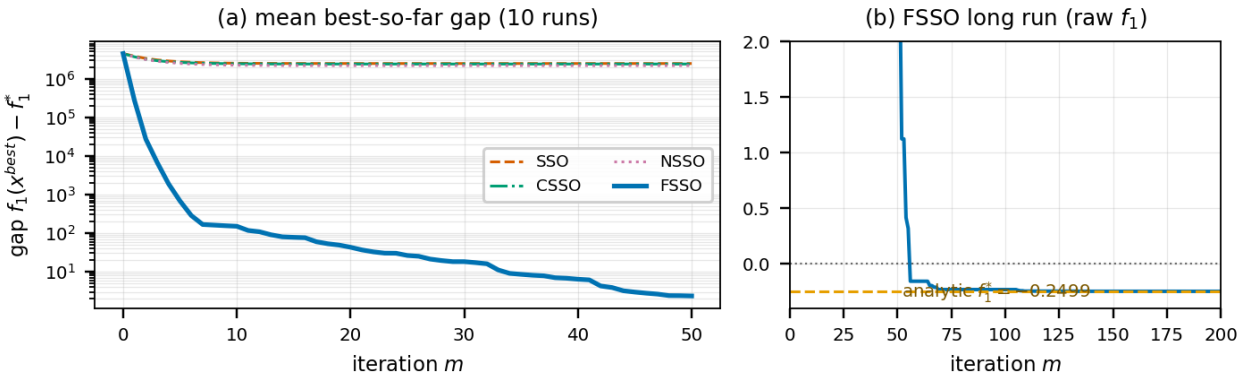


Fig. 4. Regenerated convergence on f_1: previous variants stagnate while FSSO approaches the analytic minimum.

Table 3. FSSO parameters for f_1 (rounded values from the original campaign)

Par.	Value	Par.	Value	Par.	Value
x1	-0.77×10^{-6}	x11	0.10×10^{-2}	x21	0.35×10^{-3}
x2	-0.28×10^{-3}	x12	0.14×10^{-3}	x22	-0.15×10^{-2}
x3	0.12×10^{-2}	x13	-0.11×10^{-3}	x23	-0.18×10^{-2}
x4	0.10×10^{-2}	x14	0.23×10^{-2}	x24	-0.79×10^{-2}
x5	-0.34×10^{-3}	x15	0.26×10^{-4}	x25	0.81×10^{-4}
x6	-0.22×10^{-2}	x16	-0.31×10^{-2}	x26	0.48×10^{-2}
x7	0.23×10^{-2}	x17	0.99×10^{-3}	x27	0.28×10^{-2}
x8	0.80×10^{-3}	x18	-0.84×10^{-3}	x28	0.68×10^{-4}
x9	-0.37×10^{-3}	x19	-0.13×10^{-2}	x29	0.43×10^{-3}
x10	0.67×10^{-3}	x20	-0.24×10^{-2}	x30	-0.96×10^{-3}

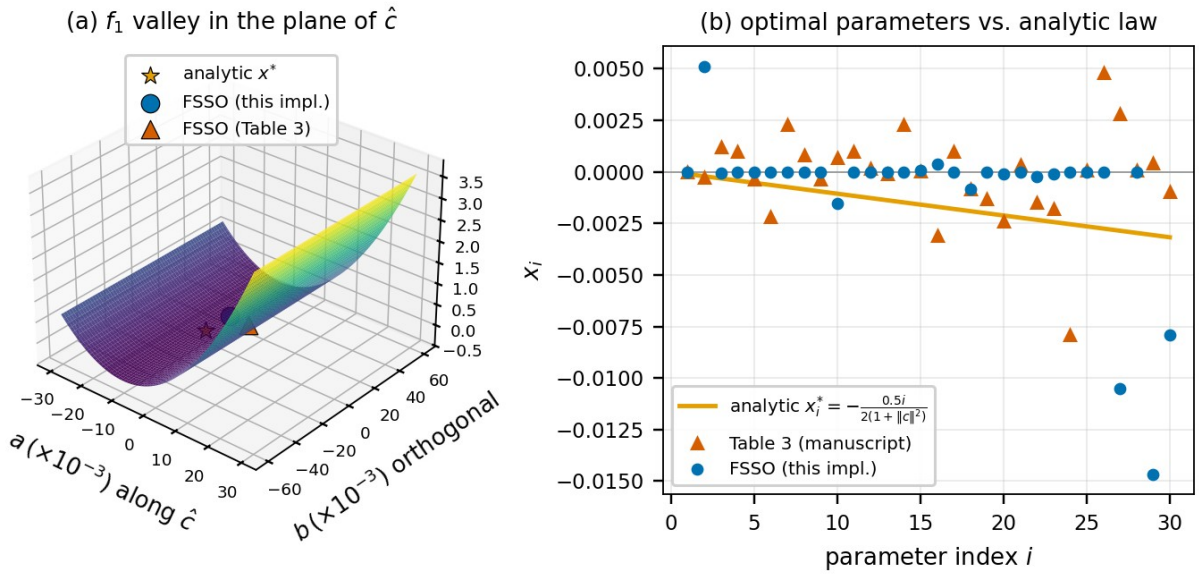


Fig. 7. Geometry of the f_1 optimum and parameter profiles versus the analytic law.

5.2 Comparison with other optimization algorithms

Table 4. Standard benchmark functions (n = 30; minimum 0)

Function	Definition	Domain
F_1 Step	$\Sigma\lfloor x_i+0.5\rfloor^2$	[−100,100]
F_2 Rastrigin	$\Sigma(x_i^2-10\cos(2\pi x_i)+10)$	[−5.12,5.12]
F_3 Ackley	$-20e^{(-0.2\sqrt{(\Sigma x_i^2/n)})} - e^{(\Sigma\cos(2\pi x_i)/n)}+20+e$	[−32,32]
F_4 Zakharov	$\Sigma x_i^2+(\Sigma 0.5 i x_i)^2+(\Sigma 0.5 i x_i)^4$	[−100,100]

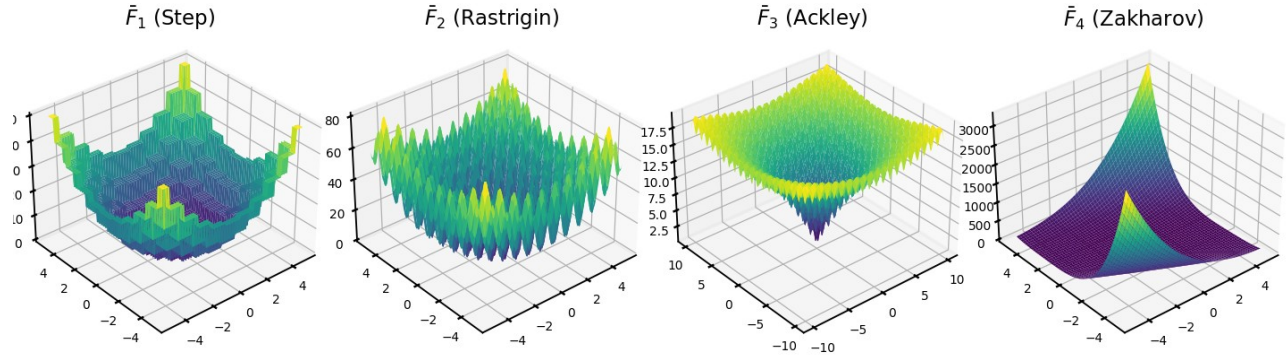


Fig. 5. Three-dimensional surfaces of the four benchmark functions (two-dimensional instances).

Table 5. Performance comparison on the Table 4 benchmarks

Algorithm	Statistic	F_1	F_2	F_3	F_4
FSSO	minimum	0	0	0.58×10^{-15}	0
FSSO	mean	0	0	0.27×10^{-13}	0
FSSO	std.	0	0	0.39×10^{-13}	0
ISOA	minimum	0	0.35×10^{-7}	0.15×10^{-3}	0
ISOA	mean	0	0.13×10^{-2}	0.54×10^{-3}	0
ISOA	std.	0	0.37×10^{-2}	0.28×10^{-3}	0
PSO	minimum	2147	170.70	10.37	0.14×10^{-2}
PSO	mean	4423.28	230.00	12.55	2.36
PSO	std.	1331.35	25.50	1.03	1.74
GA	minimum	1256	106.88	7.49	0.40×10^{-3}
GA	mean	2113.88	150.12	8.88	1.51
GA	std.	416.55	16.96	0.66	1.54
FACS	minimum	137.52	83.72	7.78	0.73×10^{-5}

Algorithm	Statistic	F_1	F_2	F_3	F_4
FACS	mean	203.37	152.79	8.21	0.06
FACS	std.	75.81	63.25	1.93	0.47×10^{-2}
DE	minimum	93	78.87	3.72	0.21×10^{-7}
DE	mean	145.80	98.48	4.86	0.67×10^{-2}
DE	std.	38.40	8.25	0.43	0.02

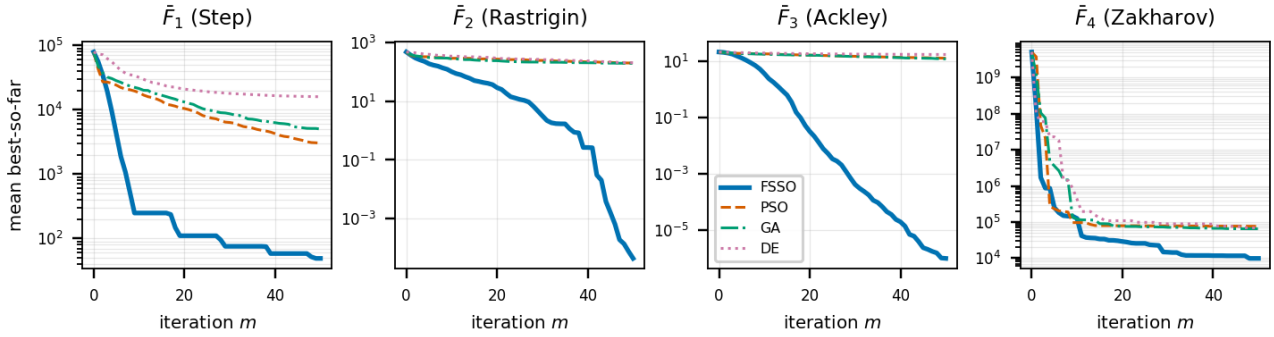


Fig. 6. Regenerated mean best-so-far convergence of FSSO, PSO, GA, and DE.

FSSO matches ISOA on the easiest functions and reaches the optimum on the highly multimodal functions where ISOA retains residual error. Gradient-driven forward movement explains the speed advantage; jumps and sign-reversing rotations improve robustness.

6. Application to Epidemic Time-Series Prediction

Daily confirmed COVID-19 counts for seven countries (USA, India, Russia, Iran, Norway, UK, and Japan) were split 80/20 into training and test data. The T–S model used five rules; FSSO used five sharks and 50 iterations. Accuracy was evaluated by RMSE, MAPE, and SMAPE.

$$MAPE = \frac{100\,\%}{T_s}\sum_{t=1}^{T_s}\left|\frac{y_t-\hat{y}_t}{y_t}\right| \tag{41}$$

$$SMAPE = \frac{100\,\%}{T_s}\sum_{t=1}^{T_s}\frac{\left|y_t-\hat{y}_t\right|}{\left(\left|y_t\right|+\left|\hat{y}_t\right|\right)/2} \tag{42}$$

Figure restoration note

Figures 9–14 below replace the blurred scans in the source PDF. They are clean schematic reconstructions digitized from the visible traces and calibrated to the quantitative results in Tables 6–8; the tables remain the authoritative numerical record.

6.1 Prediction in the training phase

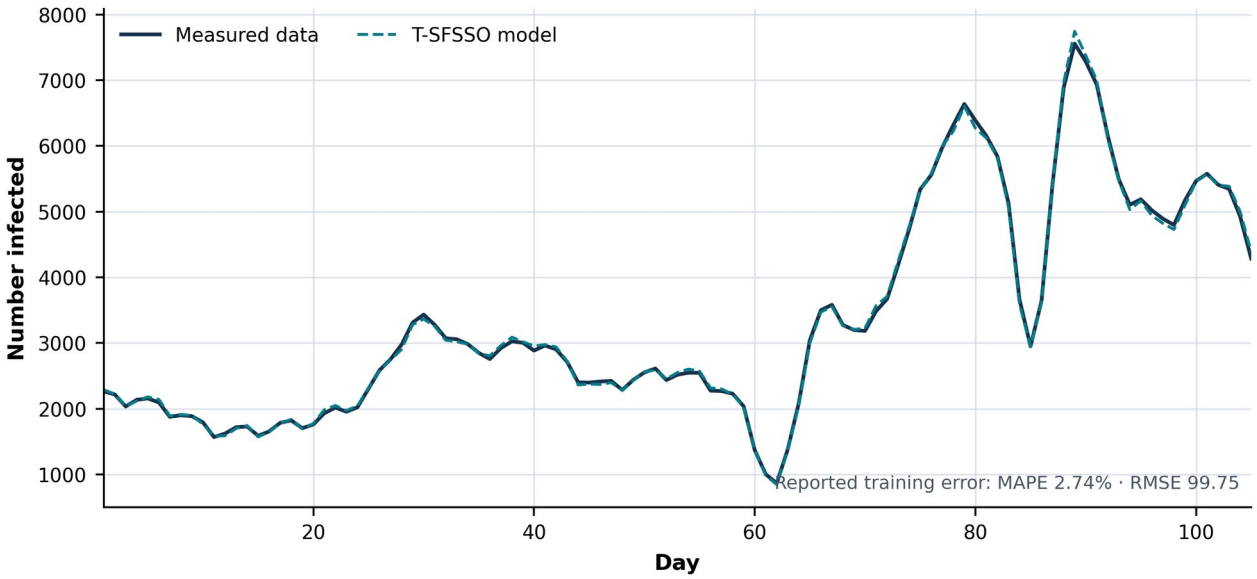


Fig. 9. Training prediction by T–SFSSO — reconstructed from the source trace and calibrated to Table 6.



Fig. 10. Training prediction by FTSOAX — reconstructed from the source trace and calibrated to Table 6.

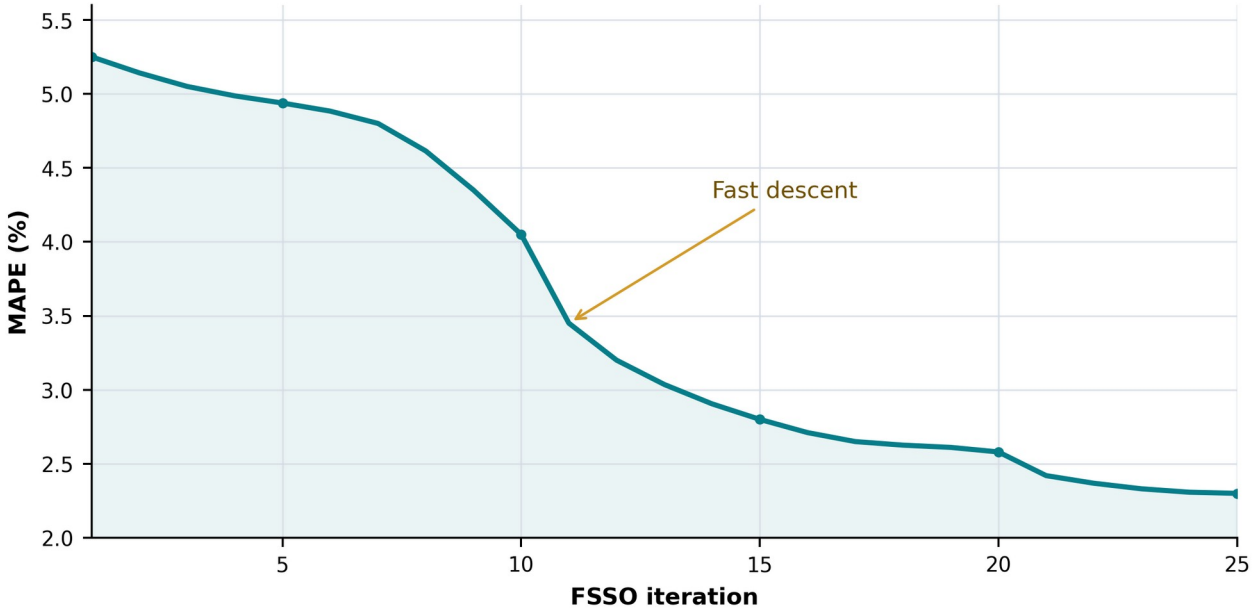


Fig. 11. Reconstructed MAPE convergence over 25 FSSO iterations, digitized from the source scan.

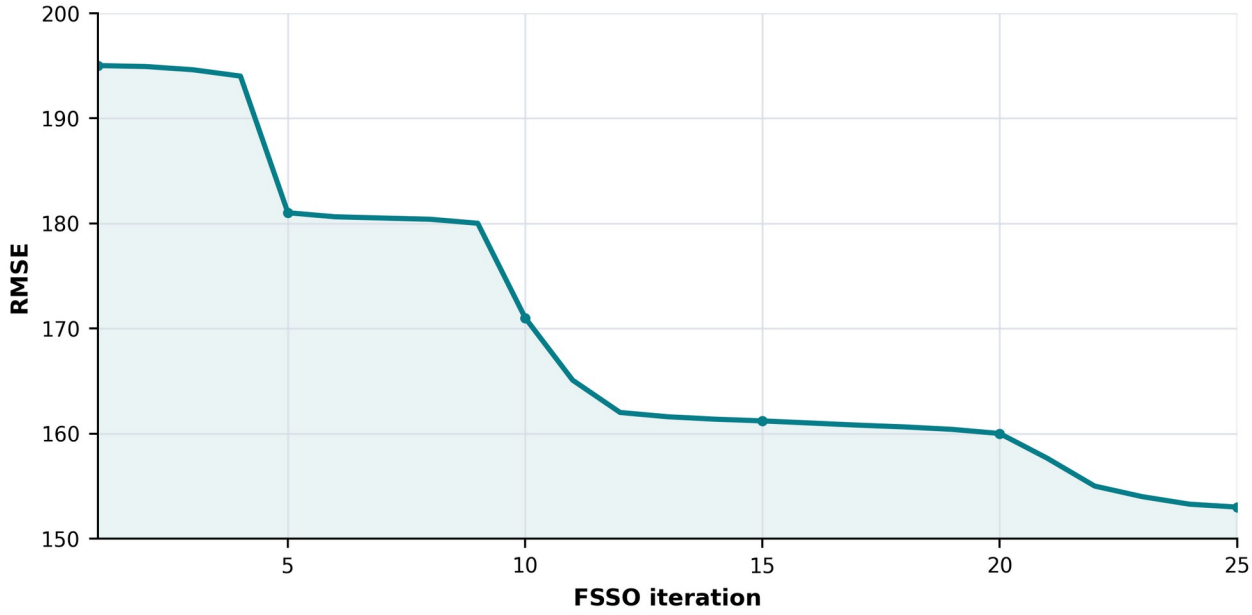


Fig. 12. Reconstructed RMSE convergence over 25 FSSO iterations, digitized from the source scan.

Table 6. Training-phase error

Algorithm	MAPE	RMSE
T–SFSSO	2.74%	99.75
FTSOAX	4.68%	213.26

Table 7. Training comparison (RMSE / SMAPE)

Country	[22]	[23]	[24]	FTSOAX	T–SFSSO
USA	21763.26 / 23.98	24518.61 / 32.76	18879.43 / 18.71	10236.83 / 15.18	9783.29 / 13.84
India	14332.20 / 29.46	13831.16 / 24.02	10915.50 / 10.17	6476.96 / 7.81	6074.28 / 7.32
Russia	1957.36 / 11.66	2133.67 / 13.75	1786.00 / 7.80	1240.49 / 3.62	1052.17 / 3.03
Iran	2556.53 / 24.65	2671.34 / 15.40	2674.77 / 10.92	1735.81 / 8.02	1625.34 / 2.18
Norway	133.82 / 40.69	141.49 / 52.33	130.50 / 36.28	76.79 / 31.63	59.76 / 26.71
UK	3167.46 / 43.44	3202.68 / 36.40	2797.58 / 16.99	1343.01 / 14.53	1293.62 / 13.28

Country	[22]	[23]	[24]	FTSOAX	T–SFSSO
Japan	1323.14 / 57.91	883.20 / 50.46	900.65 / 40.46	465.13 / 15.45	387.44 / 13.47

T–SFSSO has the smallest RMSE and SMAPE for every country in the training comparison.

6.2 Prediction for the test data

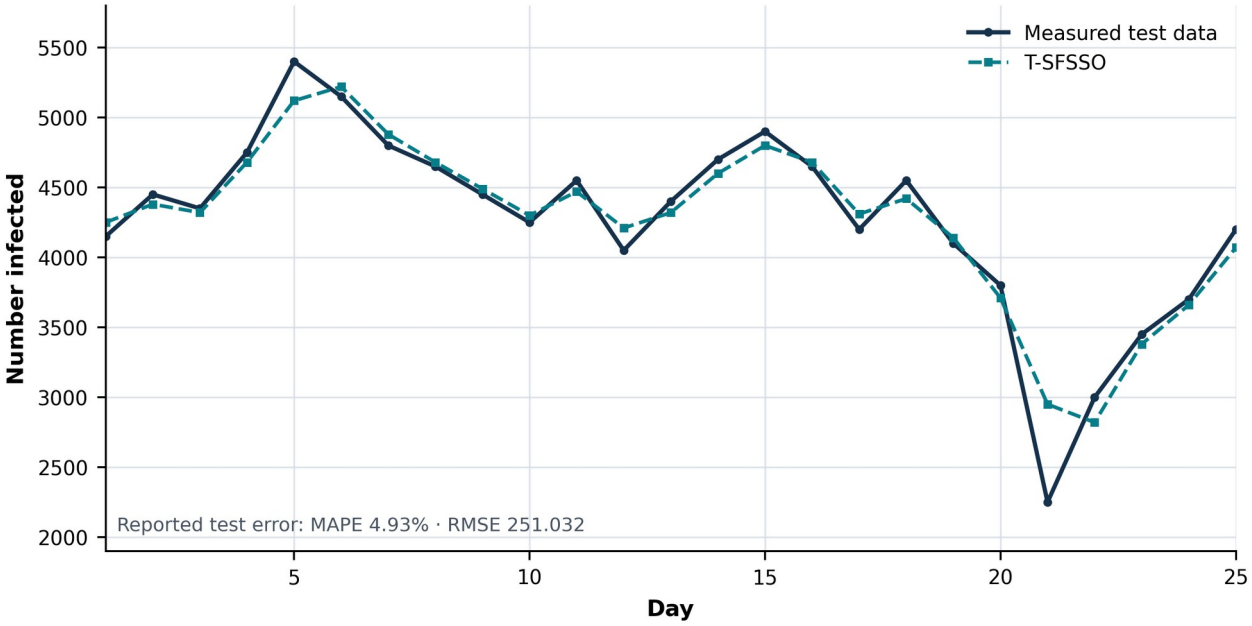


Fig. 13. Test prediction by T–SFSSO — reconstructed from the source trace and calibrated to Table 8.

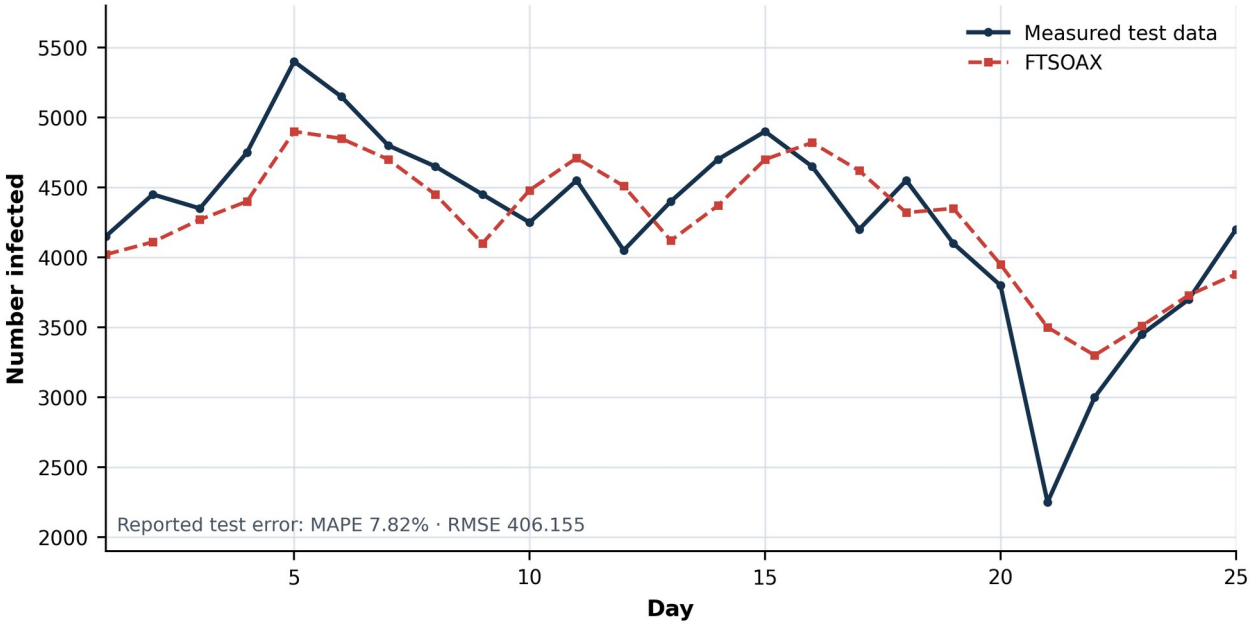


Fig. 14. Test prediction by FTSOAX — reconstructed from the source trace and calibrated to Table 8.

Table 8. Test-data error

Algorithm	MAPE	RMSE
T–SFSSO	4.93%	251.032
FTSOAX	7.82%	406.155

Table 9. Test comparison (RMSE / SMAPE)

Country	[22]	[23]	[24]	FTSOAX	T–SFSSO
USA	25142.50 / 54.59	25619.50 / 57.84	28685.24 / 54.07	23865.82 / 52.36	22761.69 / 51.87
India	13355.95 / 27.67	13526.97 / 28.00	4822.58 / 9.99	4285.90 / 7.98	3989.93 / 6.13
Russia	4349.99 / 7.42	4425.84 / 8.03	4380.58 / 7.04	4255.50 / 5.46	4109.84 / 5.26
Iran	6893.40 / 24.95	6856.67 / 24.77	7765.32 / 29.13	6292.97 / 25.38	6085.45 / 23.68
Norway	156.75 / 55.18	139.46 / 47.32	131.10 / 47.21	133.69 / 46.53	102.51 / 43.02
UK	7727.18 / 15.57	7949.36 / 16.59	7181.82 / 14.89	8515.25 / 15.58	7082.93 / 14.13
Japan	1397.51 / 36.55	1371.77 / 34.50	1520.70 / 32.90	1332.31 / 31.17	1309.83 / 31.06

Table 10. Forecast-horizon comparison (RMSE / SMAPE)

Horizon	[22]	[23]	[24]	FTSOAX	T–SFSSO
40 days	13613.67 / 28.28	13796.58 / 28.63	4935.42 / 10.17	3532.31 / 7.31	3182.35 / 6.81
100 days	15950.56 / 42.90	15787.34 / 40.66	5100.16 / 11.68	3148.61 / 6.69	2907.54 / 6.23
180 days	12368.06 / 37.36	12128.53 / 34.85	4490.23 / 15.60	2785.22 / 10.77	2513.07 / 9.82

Across countries and all three forecast horizons, T–SFSSO retains the best RMSE and SMAPE in the manuscript's comparison set.

7. Discussion: Strengths, Limitations, and Future Research

7.1 Strengths of the proposed approach

The merits demonstrated in Sections 5 and 6 can be summarized as follows. First, FSSO retains the distinctive asset of the SSO family — the use of gradient information alongside stochastic local search — and repairs the four defects that prevented that asset from paying off: the adaptive time interval accelerates the traverse of steep regions, the stochastic speed cap eliminates premature freezing, and the jump together with the sign-reversing rotation restores genuine global search (Section 3.4 makes this precise: Eq. (36) holds for FSSO but not for the conventional SSO). Second, the algorithm is theoretically grounded — convergence, stability, and complexity are established rather than assumed — and its complexity remains of the same order as the conventional SSO, which the measured runtimes confirm. Third, in the prediction pipeline the combination "RLS for a cheap, well-posed initialization + FSSO for global refinement" is decisive: it reduces the computation that a from-scratch metaheuristic identification would require and simultaneously improves accuracy, halving the RMSE of the strongest recent baseline (FTSOAX) and winning on both indices in all seven countries and at all three forecast horizons. Fourth, the benchmark analysis contributes a result of independent interest: the closed-form minimum of the weighted-sum function f_1 (Proposition 1), which corrects a value misreported in earlier literature and explains why algorithms without sign-reversing local search were structurally unable to find it. Finally, the interpretability of the T-S rule base is preserved — each rule remains a readable linear recursion — which matters in epidemiological practice, and complete Python and MATLAB implementations make every curve reproducible.

7.2 Limitations

The approach also has limitations that should be stated plainly. (i) Gradient dependence. FSSO requires the gradient of the objective; where no analytic gradient exists it falls back on central differences at a cost of 2·ND evaluations per shark and stage, and on non-differentiable or noisy objectives (e.g., the step function F_1) the gradient conveys little information and the burden of progress shifts to the rotation and jump operators. (ii) Hyper-parameters. The improvements introduce constants — C_g, C_t, Q, ε_J, T_J — whose values were chosen by experiment; although Section 3.5 shows that no admissible setting destabilizes the search, a principled adaptive tuning is still open. (iii) Scope of the evidence. The benchmark suite, while standard, is small by the

norms articulated in [62], and the statistical comparison would be strengthened by nonparametric significance tests over larger run counts. (iv) Rounding and reporting. As Fig. 7 shows, two-digit rounding of the reported optimal parameters loses most of the attained depth of the f_1 valley; full-precision parameter files should accompany published tables. (v) Modelling scope. The application uses a univariate daily-cases series; exogenous drivers that demonstrably improve epidemic forecasts — variant frequencies, vaccination, mobility [12, 13] — are not yet part of the model, the 80/20 split provides a single evaluation window rather than a rolling-origin design, and reporting artefacts of surveillance data (weekend effects, revisions) are not treated explicitly.

7.3 Directions for future research

Several directions follow naturally. First, self-adaptive control of the new constants — e.g., success-based adaptation of Q and C_t, or their evolution alongside the population — would remove the remaining manual tuning. Second, surrogate or analytic gradients for the fuzzy identification objective would cut the dominant cost term c_∇ of Eq. (39). Third, a multi-objective FSSO (accuracy versus rule-base compactness) would connect this work to the multi-objective SSO line of [66], and interval type-2 antecedents could improve robustness under the heavy noise of surveillance data. Fourth, hybridization with deep sequence models — using T-SFSSO as an interpretable front end and an LSTM/GRU residual learner [10, 11] — and the inclusion of exogenous covariates [12] are promising for accuracy. Fifth, the theoretical program can be extended: finite-time convergence rates for FSSO in the spirit of recent swarm-convergence results [63], and formal Lyapunov certificates for the identified dynamic T-S predictor. Finally, broader empirical validation — rolling-origin evaluation across full pandemic waves, additional countries, other epidemic and non-epidemic series (energy load, wind power, finance), and the modern benchmark suites urged by [62] — would consolidate the evidence.

8. Conclusions

This paper proposed T-SFSSO, a new prediction method for epidemic time series, built on a dynamic Takagi–Sugeno fuzzy model identified by an improved shark smell optimization algorithm (FSSO). The improvement of the optimizer is not a single heuristic but four coordinated repairs of documented defects of the SSO family: a probabilistic, gradient-driven adaptive time interval that raised the search speed by enlarging the scope of the shark's forward movement; a gradient-triggered jump that supplies escape points in local-optimal intervals and thereby sharply lowered the rate of terminations at local optima; a corrected, stochastic speed-limit constant on (0, 2) that removed the systematic velocity decay of earlier variants; and an extended rotational coefficient on (−1.3, 1.3) that made sign-reversing

On standard benchmarks FSSO outperformed the conventional SSO, its chaotic and normal-distribution variants, and PSO, GA, DE, FACS, and the recent ISOA — and it discovered that the minimum of a widely used weighted-sum benchmark is negative, a fact proved here in closed form ($f_1^* = -w/(4(w+1)) \approx -0.2499$) and previously unreported. In the application, the parameter identification of the dynamic T-S fuzzy model is achieved with high accuracy by initializing with recursive least squares and refining with FSSO; on daily confirmed COVID-19 cases of seven countries the proposed T-SFSSO reduced the training MAPE from 4.68% to 2.74% and the test MAPE from 7.82% to 4.93% relative to the recent FTSOAX model, roughly halved the RMSE in both phases, and attained the best RMSE and SMAPE among all compared models in every country and at every forecast horizon examined. As a result, the proposed T-SFSSO demonstrated its performance in practice compared with several other prediction models, while retaining the interpretability of a rule-based model and a computational cost linear in all design dimensions. The limitations and the research agenda that follows from them — adaptive hyper-parameter control, surrogate gradients, multi-objective and type-2 extensions, deep hybrid architectures, and broader rolling-origin validation — were laid out in Section 7.

References

1. Marques, J. A. L., Gois, F. N. B., Xavier-Neto, J., Fong, S. J. (2021). Predictive Models for Decision Support in the COVID-19 Crisis. SpringerBriefs in Applied Sciences and Technology. Springer, Cham. https://doi.org/10.1007/978-3-030-61913-8

2. Feng, S., Feng, Z., Ling, C., Chang, C., Feng, Z. (2021). Prediction of the COVID-19 epidemic trends based on SEIR and AI models. PLoS ONE, 16(1), e0245101.

3. Cai, M., Karniadakis, G. E., Li, C. (2022). Fractional SEIR model and data-driven predictions of COVID-19 dynamics of Omicron variant. Chaos, 32(7), 071101.

4. Kiarie, J., Mwalili, S., Mbogo, R. (2022). Forecasting the spread of the COVID-19 pandemic in Kenya using SEIR and ARIMA models. Infectious Disease Modelling, 7(2), 179–188.

5. de Araújo Morais, L. R., da Silva Gomes, G. S. (2022). Forecasting daily Covid-19 cases in the world with a hybrid ARIMA and neural network model. Applied Soft Computing, 126, 109315.

6. Thorakkattle, M. N., Farhin, S., Khan, A. A. (2022). Forecasting the trends of Covid-19 and causal impact of vaccines using Bayesian structural time series and ARIMA. Annals of Data Science, 9(5), 1025–1047.

7. ArunKumar, K. E., Kalaga, D. V., Kumar, Ch. M. S., Chilkoor, G., Kawaji, M., Brenza, T. M. (2021). Forecasting the dynamics of cumulative COVID-19 cases (confirmed, recovered and deaths) for top-16 countries using statistical machine learning models: Auto-Regressive Integrated Moving Average (ARIMA) and Seasonal Auto-Regressive Integrated Moving Average (SARIMA). Applied Soft Computing, 103, 107161.

8. Masum, M., Shahriar, H., Haddad, H. M., Alam, M. S. (2020). r-LSTM: Time series forecasting for COVID-19 confirmed cases with LSTM-based framework. Proceedings of the 2020 IEEE International Conference on Big Data, 1374–1379.

9. Hochreiter, S., Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780.

10. ArunKumar, K. E., Kalaga, D. V., Kumar, Ch. M. S., Kawaji, M., Brenza, T. M. (2022). Comparative analysis of Gated Recurrent Units (GRU), long Short-Term Memory (LSTM) cells, Autoregressive Integrated Moving Average (ARIMA), Seasonal Autoregressive Integrated Moving Average (SARIMA) for forecasting COVID-19 trends. Alexandria Engineering Journal, 61(10), 7585–7603.

11. Gautam, Y. (2022). Transfer learning for COVID-19 cases and deaths forecast using LSTM network. ISA Transactions, 124, 41–56.

12. Du, H., Dong, E., Badr, H. S., Petrone, M. E., Grubaugh, N. D., Gardner, L. M. (2023). Incorporating variant frequencies data into short-term forecasting for COVID-19 cases and deaths in the USA: a deep learning approach. eBioMedicine, 89, 104482.

13. Cramer, E. Y., Ray, E. L., Lopez, V. K., et al. (2022). Evaluation of individual and ensemble probabilistic forecasts of COVID-19 mortality in the United States. Proceedings of the National Academy of Sciences, 119(15), e2113561119. Nixon, K., Jindal, S., Parker, F., Reich, N. G., Ghobadi, K., Lee, E. C.,

14. Truelove, S., Gardner, L. (2022). An evaluation of prospective COVID-19 modelling studies in the USA: from data to science translation. The Lancet Digital Health, 4(10), e738–e747.

15. Rahimi, I., Chen, F., Gandomi, A. H. (2023). A review on COVID-19 forecasting models. Neural Computing and Applications, 35(33), 23671–23681.

16. Alyasseri, Z. A. A., Al-Betar, M. A., Doush, I. A., Awadallah, M. A., Abasi, A. K., Makhadmeh, S. N., Alomari, O. A., Abdulkareem, K. H., Adam, A., Damasevicius, R., Mohammed, M. A., Zitar, R. A. (2022). Review on COVID-19 diagnosis models based on machine learning and deep learning approaches. Expert Systems, 39(3), e12759.

17. Kumar, N., Susan, S. (2021). Particle swarm optimization of partitions and fuzzy order for fuzzy time series forecasting of COVID-19. Applied Soft Computing, 110, 107611.

18. Ceylan, Z. (2021). Short-term prediction of COVID-19 spread using grey rolling model optimized by particle swarm optimization. Applied Soft Computing, 109, 107592.

19. Xian, S., Chen, K., Cheng, Y. (2022). Improved seagull optimization algorithm of partition and XGBoost of prediction for fuzzy time series forecasting of COVID-19 daily confirmed. Advances in Engineering Software, 173, 103212.

20. Efendi, R., Ismail, Z., Deris, M. M. (2013). Improved weight fuzzy time series as used in the exchange rates forecasting of US dollar to ringgit Malaysia. International Journal of Computational Intelligence and Applications, 12(1), 1350005.

21. Jiang, P., Yang, H., Heng, J. (2019). A hybrid forecasting system based on fuzzy time series and multi-objective optimization for wind speed forecasting. Applied Energy, 235, 786–801.

22. Zhang, W., Zhang, S., Zhang, S. (2019). A novel method based on FTS with both GA-FCM and multifactor BPNN for stock forecasting. Soft Computing, 23(16), 6979–6994.

23. Sadaei, H. J., Enayatifar, R., Abdullah, A. H., Gani, A. (2014). Short-term load forecasting using a hybrid model with a refined exponentially weighted fuzzy time series and an improved harmony search. International Journal of Electrical Power & Energy Systems, 62, 118–129.

24. Kumar, N., Kumar, H. (2022). A novel hybrid fuzzy time series model for prediction of COVID-19 infected cases and deaths in India. ISA Transactions, 124, 69–81. Cansu, T., Bas, E., Egrioglu, E. (2026). Mixed order single variable

25. intuitionistic fuzzy time series forecasting method based on a new artificial neural network and grey wolf optimization algorithm. Scientific Reports, 16, 15682.
26. Al-Qaness, M. A. A., Ewees, A. A., Fan, H., Abualigah, L., Abd Elaziz, M. (2022). Boosted ANFIS model using augmented marine predator algorithm with mutation operators for wind power forecasting. Applied Energy, 314, 118851.
27. Mamdani, E. H., Assilian, S. (1975). An experiment in linguistic synthesis with a fuzzy logic controller. International Journal of Man-Machine Studies, 7(1), 1–13.
28. Takagi, T., Sugeno, M. (1985). Fuzzy identification of systems and its applications to modeling and control. IEEE Transactions on Systems, Man, and Cybernetics, 15(1), 116–132.
29. Gao, Y., Er, M. J. (2005). NARMAX time series model prediction: feedforward and recurrent fuzzy neural network approaches. Fuzzy Sets and Systems, 150(2), 331–350.
30. Juang, C.-F., Lin, C.-T. (1998). An online self-constructing neural fuzzy inference network and its applications. IEEE Transactions on Fuzzy Systems, 6(1), 12–32.
31. Jang, J.-S. R. (1993). ANFIS: adaptive-network-based fuzzy inference system. IEEE Transactions on Systems, Man, and Cybernetics, 23(3), 665 –685.
32. Chen, Y. M., Lin, C. T. (2007). Dynamic parameter optimization of evolutionary computation for on-line prediction of time series with changing dynamics. Applied Soft Computing, 7(4), 1170–1176.
33. Kukolj, D. (2002). Design of adaptive Takagi–Sugeno–Kang fuzzy models. Applied Soft Computing, 2(2), 89–103.
34. Araujo, E., Coelho, L. d. S. (2008). Particle swarm approaches using Lozi map chaotic sequences to fuzzy modelling of an experimental thermal-vacuum system. Applied Soft Computing, 8(4), 1354–1364.
35. Papadakis, S. E., Theocharis, J. B. (2002). A GA-based fuzzy modeling approach for generating TSK models. Fuzzy Sets and Systems, 131(2), 121–152.
36. Salimi-Badr, A., Ebadzadeh, M. M. (2022). A novel learning algorithm based on computing the rules' desired outputs of a TSK fuzzy neural network with non-separable fuzzy rules. Neurocomputing, 470, 139–153.
37. Deif, M., Hammam, R., Solyman, A. (2021). Adaptive neuro-fuzzy inference system (ANFIS) for rapid diagnosis of COVID-19 cases based on routine blood tests. International Journal of Intelligent Engineering and Systems, 14(2), 178–189.
38. Kennedy, J., Eberhart, R. (1995). Particle swarm optimization. Proceedings of ICNN'95 — International Conference on Neural Networks, 4, 1942–1948.
39. Goldberg, D. E. (1989). Genetic Algorithms in Search, Optimization and Machine Learning. Addison-Wesley, Reading, MA.
40. Storn, R., Price, K. (1997). Differential evolution — a simple and efficient heuristic for global optimization over continuous spaces. Journal of Global Optimization, 11(4), 341–359.
41. Tao, C.-W., Taur, J.-S., Jeng, J.-T., Wang, W.-Y. (2009). A novel fuzzy ant colony system for parameter determination of fuzzy controllers. International Journal of Fuzzy Systems, 11(4), 298–307.
42. Abualigah, L., Abd Elaziz, M., Sumari, P., Geem, Z. W., Gandomi, A. H. (2022). Reptile Search Algorithm (RSA): a nature-inspired meta-heuristic optimizer. Expert Systems with Applications, 191, 116158.
43. Agushaka, J. O., Ezugwu, A. E., Abualigah, L. (2022). Dwarf mongoose optimization algorithm. Computer Methods in Applied Mechanics and Engineering, 391, 114570.
44. Ezugwu, A. E., Agushaka, J. O., Abualigah, L., Mirjalili, S., Gandomi, A. H. (2022). Prairie dog optimization algorithm. Neural Computing and Applications, 34(22), 20017–20065.
45. Hashim, F. A., Hussien, A. G. (2022). Snake Optimizer: a novel meta-heuristic optimization algorithm. Knowledge-Based Systems, 242, 108320.
46. Zhao, S., Zhang, T., Ma, S., Chen, M. (2022). Dandelion Optimizer: a nature-inspired metaheuristic algorithm for engineering applications. Engineering Applications of Artificial Intelligence, 114, 105075.
47. Zhao, W., Wang, L., Mirjalili, S. (2022). Artificial hummingbird algorithm: a new bio-inspired optimizer with its engineering applications. Computer Methods in Applied Mechanics and Engineering, 388, 114194.
48. Wang, L., Cao, Q., Zhang, Z., Mirjalili, S., Zhao, W. (2022). Artificial rabbits optimization: a new bio-inspired meta-heuristic algorithm for solving engineering optimization problems. Engineering Applications of Artificial Intelligence, 114, 105082.
49. Chopra, N., Ansari, M. M. (2022). Golden jackal optimization: a novel nature-inspired optimizer for engineering applications. Expert Systems with Applications, 198, 116924.
50. Trojovský, P., Dehghani, M. (2022). Pelican optimization algorithm: a novel nature-inspired algorithm for engineering applications. Sensors, 22(3), 855. Zamani, H., Nadimi-Shahraki, M. H., Gandomi, A. H. (2022). Starling
51. murmuration optimizer: a novel bio-inspired algorithm for global and engineering optimization. Computer Methods in Applied Mechanics and Engineering, 392, 114616.

52. Oyelade, O. N., Ezugwu, A. E., Mohamed, T. I. A., Abualigah, L. (2022). Ebola optimization search algorithm: a new nature-inspired metaheuristic optimization algorithm. IEEE Access, 10, 16150–16177.

53. Ahmadianfar, I., Heidari, A. A., Noshadian, S., Chen, H., Gandomi, A. H. (2022). INFO: an efficient optimization algorithm based on weighted mean of vectors. Expert Systems with Applications, 195, 116516.

54. Dehghani, M., Montazeri, Z., Trojovská, E., Trojovský, P. (2023). Coati optimization algorithm: a new bio-inspired metaheuristic algorithm for solving optimization problems. Knowledge-Based Systems, 259, 110011.

55. Abdel-Basset, M., Mohamed, R., Azeem, S. A. A., Jameel, M., Abouhawwash, M. (2023). Kepler optimization algorithm: a new metaheuristic algorithm inspired by Kepler's laws of planetary motion. Knowledge-Based Systems, 268, 110454.

56. Abdel-Basset, M., Mohamed, R., Jameel, M., Abouhawwash, M. (2023). Nutcracker optimizer: a novel nature-inspired metaheuristic algorithm for global optimization and engineering design problems. Knowledge-Based Systems, 262, 110248.

57. Seyyedabbasi, A., Kiani, F. (2023). Sand Cat swarm optimization: a nature-inspired algorithm to solve global optimization problems. Engineering with Computers, 39(4), 2627–2651.

58. Su, H., Zhao, D., Heidari, A. A., Liu, L., Zhang, X., Mafarja, M., Chen, H. (2023). RIME: a physics-based optimization. Neurocomputing, 532, 183–214.

59. Agushaka, J. O., Ezugwu, A. E., Abualigah, L. (2023). Gazelle optimization algorithm: a novel nature-inspired metaheuristic optimizer. Neural Computing and Applications, 35(5), 4099–4131.

60. Amiri, M. H., Mehrabi Hashjin, N., Montazeri, M., Mirjalili, S., Khodadadi, N. (2024). Hippopotamus optimization algorithm: a novel nature-inspired optimization algorithm. Scientific Reports, 14, 5032.

61. Lian, J., Hui, G., Ma, L., Zhu, T., Wu, X., Heidari, A. A., Chen, Y., Chen, H. (2024). Parrot optimizer: algorithm and applications to medical problems. Computers in Biology and Medicine, 172, 108064.

62. Kudela, J. (2022). A critical problem in benchmarking and analysis of evolutionary computation methods. Nature Machine Intelligence, 4(12), 1238–1245.

63. Huang, H., Qiu, J., Riedl, K. (2023). On the global convergence of particle swarm optimization methods. Applied Mathematics & Optimization, 88, 30.

64. Abedinia, O., Amjady, N., Ghasemi, A. (2016). A new metaheuristic algorithm based on shark smell optimization. Complexity, 21(5), 97–116.

65. Abedinia, O., Amjady, N. (2015). Short-term wind power prediction based on hybrid neural network and chaotic shark smell optimization. International Journal of Precision Engineering and Manufacturing-Green Technology, 2(3), 245–254.

66. Wang, L.-D., Wang, X.-C., Liu, K.-W. (2020). An improved multiobjective shark smell optimization algorithm for automatic train operation based on angle cosine and fusion distance. Journal of Computers, 31(4), 141–156.

67. Nasir, I. M., Raza, M., Shah, J. H., Khan, M. A., Nam, Y.-C., Nam, Y. (2023). Improved shark smell optimization algorithm for human action recognition. Computers, Materials & Continua, 76(3), 2667–2684.

68. Braik, M., Hammouri, A., Atwan, J., Al-Betar, M. A., Awadallah, M. A. (2022). White Shark Optimizer: a novel bio-inspired meta-heuristic algorithm for global optimization problems. Knowledge-Based Systems, 243, 108457.

69. Hu, G., Guo, Y., Wei, G., Abualigah, L. (2023). Genghis Khan shark optimizer: a novel nature-inspired algorithm for engineering optimization. Advanced Engineering Informatics, 58, 102210.

Appendix A. Python Reference Implementation

Complete self-contained Python implementation of FSSO and the benchmark suite. This repairs the truncated Appendix A in the source PDF export.

```
"""Reference implementation of the FSSO algorithm described in Algorithm 3."""


from __future__ import annotations


import numpy as np



def f1_table1(x: np.ndarray) -> float:
    i = np.arange(1, x.size + 1, dtype=float)
    s = np.sum(0.5 * i * x)
    return float(np.sum(x * x) + s * s + s)



def f2_matyas(x: np.ndarray) -> float:
    return float(0.26 * (x[0] ** 2 + x[1] ** 2) - 0.48 * x[0] * x[1])
```

```
def F1_step(x: np.ndarray) -> float:
    return float(np.sum(np.floor(x + 0.5) ** 2))



def F2_rastrigin(x: np.ndarray) -> float:
    return float(np.sum(x * x - 10 * np.cos(2 * np.pi * x) + 10))



def F3_ackley(x: np.ndarray) -> float:
    n = x.size
    return float(
        -20 * np.exp(-0.2 * np.sqrt(np.sum(x * x) / n))
        - np.exp(np.sum(np.cos(2 * np.pi * x)) / n)
        + 20
        + np.e
    )



def F4_zakharov(x: np.ndarray) -> float:
    i = np.arange(1, x.size + 1, dtype=float)
    s = np.sum(0.5 * i * x)
    return float(np.sum(x * x) + s * s + s**4)



def central_gradient(f, x: np.ndarray, lo: np.ndarray, hi: np.ndarray) -> np.ndarray:
    """Bound-aware central-difference gradient."""
    scale = np.maximum(1.0, np.abs(x))
    h = 1e-6 * scale
    grad = np.zeros_like(x, dtype=float)
    for j in range(x.size):
        xp = x.copy()
        xm = x.copy()
        xp[j] = min(hi[j], x[j] + h[j])
        xm[j] = max(lo[j], x[j] - h[j])
        width = xp[j] - xm[j]
        grad[j] = 0.0 if width == 0 else (f(xp) - f(xm)) / width
    return grad



def fsso(
    f,
    nd: int,
    lo,
    hi,
    *,
    NP: int = 10,
    M: int = 50,
    L: int = 5,
    mu: float = 0.9,
    alpha: float = 0.5,
    Cg: float = 1.0,
    Ct: float = 2.0,
    Q: float = 0.7,
    epsJ: float = 1e-6,
    Tjump: int = 10,
    seed: int | None = None,
):
    """Minimize ``f`` with the four FSSO improvements from the manuscript."""
    rng = np.random.default_rng(seed)
    lo = np.broadcast_to(np.asarray(lo, dtype=float), (nd,)).copy()
    hi = np.broadcast_to(np.asarray(hi, dtype=float), (nd,)).copy()
    if np.any(hi <= lo):
        raise ValueError("Every upper bound must exceed its lower bound.")


    X = lo + (hi - lo) * rng.random((NP, nd))
    V = 0.1 * (hi - lo) * (2 * rng.random((NP, nd)) - 1)
    vmax = 0.5 * (hi - lo)
    fx = np.array([f(x) for x in X], dtype=float)
    best_index = int(np.argmin(fx))
    best_x = X[best_index].copy()
    best_f = float(fx[best_index])
    history = np.empty(M + 1, dtype=float)
    history[0] = best_f


    for stage in range(1, M + 1):
        gamma = rng.uniform(0.0, 2.0)
        for p in range(NP):
            # Minimize f by moving along -grad(f), equivalent to maximizing -f.
            g = central_gradient(f, X[p], lo, hi)
            G = float(np.linalg.norm(g))
            trial_velocity = -mu * rng.random(nd) * g + alpha * rng.random(nd) * V[p]
            former = np.maximum(np.abs(V[p]), 1e-12)
            V[p] = np.sign(trial_velocity) * np.minimum(np.abs(trial_velocity), gamma * former)
            V[p] = np.clip(V[p], -vmax, vmax)


            if G < Cg or rng.random() < Q:
                dt = 1.0
            else:
                dt = (G / Cg) * Ct * rng.random()
            z = np.clip(X[p] + V[p] * dt, lo, hi)


            if stage % Tjump == 0 and G < epsJ:
                z = lo + (hi - lo) * rng.random(nd)
```

```
            candidates = [z]
            for _ in range(L):
                R3 = rng.uniform(-1.3, 1.3, size=nd)
                candidates.append(np.clip(z + R3 * z, lo, hi))
            candidate_values = np.array([f(c) for c in candidates], dtype=float)
            winner = int(np.argmin(candidate_values))
            if candidate_values[winner] <= fx[p]:
                X[p] = candidates[winner]
                fx[p] = candidate_values[winner]


            if fx[p] < best_f:
                best_f = float(fx[p])
                best_x = X[p].copy()
        history[stage] = best_f


    return best_x, best_f, history



if __name__ == "__main__":
    nd = 30
    xb, fb, hist = fsso(f1_table1, nd, -5.0, 30.0, M=200, seed=7)
    w = np.sum((0.5 * np.arange(1, nd + 1)) ** 2)
    print(f"FSSO best f1 = {fb:.6f}; analytic minimum = {-w / (4 * (w + 1)):.6f}")


    suite = [
        ("Step", F1_step, -100.0, 100.0),
        ("Rastrigin", F2_rastrigin, -5.12, 5.12),
        ("Ackley", F3_ackley, -32.0, 32.0),
        ("Zakharov", F4_zakharov, -100.0, 100.0),
    ]
    for name, function, lower, upper in suite:
        _, value, _ = fsso(function, nd, lower, upper, M=50, seed=1)
        print(f"FSSO on {name:10s}: best = {value:.4g}")
```

Appendix B. MATLAB Reference Implementation

Complete MATLAB R2016b+ implementation mirroring Appendix A. This repairs the truncated Appendix B in the source PDF export.

```
%% FSSO reference implementation (Algorithm 3)
% MATLAB R2016b or later; functions are defined below the script body.
clear; clc; rng(7);


nd = 30; lo = -5; hi = 30;
opts = struct('NP',10,'M',200,'L',5,'mu',0.9,'alpha',0.5, ...
              'Cg',1,'Ct',2,'Q',0.7,'epsJ',1e-6,'Tjump',10);
[xb, fb, hist] = fsso(@f1_table1, nd, lo, hi, opts); %#ok<ASGLU>
w = sum((0.5*(1:nd)).^2);
fA = -w/(4*(w+1));
fprintf('FSSO best f1 = %.6f; analytic minimum = %.6f\n', fb, fA);


semilogy(0:opts.M, max(hist - fA, eps), 'LineWidth', 1.4);
grid on; xlabel('iteration m'); ylabel('optimality gap');


%% Benchmark suite from Table 4
funs = {@F1_step, @F2_rastrigin, @F3_ackley, @F4_zakharov};
doms = [-100 100; -5.12 5.12; -32 32; -100 100];
names = {'Step','Rastrigin','Ackley','Zakharov'};
opts.M = 50;
for k = 1:4
    [~, value] = fsso(funs{k}, nd, doms(k,1), doms(k,2), opts);
    fprintf('FSSO on %-10s: best = %.4g\n', names{k}, value);
end


function [best_x, best_f, hist] = fsso(f, ND, lo, hi, o)
    lo = lo * ones(1, ND); hi = hi * ones(1, ND);
    X = lo + (hi-lo).*rand(o.NP, ND);
    V = 0.1*(hi-lo).*(2*rand(o.NP, ND)-1);
    vmax = 0.5*(hi-lo);
    fx = zeros(o.NP,1);
    for p = 1:o.NP, fx(p) = f(X(p,:)); end
    [best_f, idx] = min(fx); best_x = X(idx,:);
    hist = zeros(o.M+1,1); hist(1) = best_f;


    for stage = 1:o.M
        gamma = 2*rand;                         % Eq. (31)
        for p = 1:o.NP
            g = central_gradient(f, X(p,:), lo, hi);
            G = norm(g);
            trial = -o.mu*rand(1,ND).*g + o.alpha*rand(1,ND).*V(p,:);
            former = max(abs(V(p,:)), 1e-12);
            V(p,:) = sign(trial).*min(abs(trial), gamma*former); % Eq. (22)
            V(p,:) = max(-vmax, min(vmax, V(p,:)));


            if G < o.Cg || rand < o.Q
                dt = 1;
            else
                dt = (G/o.Cg)*o.Ct*rand;        % Eqs. (26)-(28)
            end
            z = max(lo, min(hi, X(p,:) + V(p,:)*dt));
```

```
            if mod(stage,o.Tjump)==0 && G < o.epsJ
                z = lo + (hi-lo).*rand(1,ND);   % Eqs. (29)-(30)
            end


            C = zeros(o.L+1,ND); C(1,:) = z;
            for ell = 1:o.L
                R3 = -1.3 + 2.6*rand(1,ND);
                C(ell+1,:) = max(lo, min(hi, z + R3.*z)); % Eq. (32)
            end
            cv = zeros(o.L+1,1);
            for ell = 1:o.L+1, cv(ell) = f(C(ell,:)); end
            [candidate_f, winner] = min(cv);
            if candidate_f <= fx(p)
                X(p,:) = C(winner,:); fx(p) = candidate_f;
            end
            if fx(p) < best_f
                best_f = fx(p); best_x = X(p,:);
            end
        end
        hist(stage+1) = best_f;
    end
end


function g = central_gradient(f, x, lo, hi)
    h = 1e-6*max(1,abs(x)); g = zeros(size(x));
    for j = 1:numel(x)
        xp = x; xm = x;
        xp(j) = min(hi(j), x(j)+h(j));
        xm(j) = max(lo(j), x(j)-h(j));
        width = xp(j)-xm(j);
        if width == 0, g(j) = 0; else, g(j) = (f(xp)-f(xm))/width; end
    end
end


function y = f1_table1(x)
    i = 1:numel(x); s = sum(0.5*i.*x);
    y = sum(x.^2) + s^2 + s;
end


function y = F1_step(x), y = sum(floor(x+0.5).^2); end
function y = F2_rastrigin(x), y = sum(x.^2-10*cos(2*pi*x)+10); end
function y = F3_ackley(x)
    n = numel(x);
    y = -20*exp(-0.2*sqrt(sum(x.^2)/n))-exp(sum(cos(2*pi*x))/n)+20+exp(1);
end
function y = F4_zakharov(x)
    i = 1:numel(x); s = sum(0.5*i.*x);
    y = sum(x.^2) + s^2 + s^4;
end
```